Flock Energy — Engineering Take-Home Submission

Urja Meter Ops: Modern REST API Gateway & Fleet Intelligence

Candidate: Anshika Sharma

Email: anshikasharma15122005@gmail.com

GitHub Repository: github.com/anshika660/flock-energy-urja-ops

Stack: Python 3.11, FastAPI, Pydantic v2, HTTPX, Pytest

1. Executive Summary & Solution Architecture

This project replaces manual browser copy-pasting from the legacy **Urja Meter Ops** portal (https://urja-ops.flockenergy.tech) with a high-performance, production-ready Python FastAPI service. The gateway reverse-engineers the legacy portal's authentication and data pipelines, resolves historical schema incompatibilities, self-heals corrupted network hierarchies, and exposes sub-millisecond multi-attribute and geospatial proximity queries alongside an interactive web dashboard.

2. Reverse-Engineering Discoveries (PROTOCOL.md Summary)

- **SvelteKit CSRF Protection:** The portal rejects cross-site form submissions with 403 Forbidden unless matching Origin and Referer headers are provided. Our client maintains an automated session with better-auth cookies and transparent token renewal.
- **Dynamic HMAC-SHA256 Bulk Ingestion:** Reverse-engineered the internal /portal/export endpoint used by the UI's 'Export all meters' button. By fetching the dynamic signing secret from /portal/keys and computing an HMAC-SHA256 signature (method\npath\nquery\ntimestamp), our gateway synchronizes all 403 meters in a single HTTP request in **0.89 seconds**.
- **Dual Legacy vs. v2 Schemas:** The portal stores meter nameplates in two incompatible schemas: legacy meters store parameters as key-value objects in detail.data, while v2 meters store an escaped JSON string in detail.classData.installed_meter. Our service unifies both into a single typed Pydantic model.
- **Self-Healing 22 Corrupted Records:** 14 meters had an entity name (e.g. 'Circle 6', 'Feeder 5') with an empty code, while 8 meters had a code (e.g. 'C-01', 'F-003') with an empty name. Our gateway learns a bidirectional canonical dictionary across the fleet and automatically heals 100% of these records on ingestion.

3. Core API Endpoints & Capabilities

Endpoint	Method	Description & Capabilities
`/api/v1/meters`	GET	Multi-attribute filterable search (by make, phase, status, DT, feeder, build, query) with sorting & pagination.
`/api/v1/meters/nearby`	GET	Geospatial radius search using Haversine distance formula (lat, lng, radius_km, limit).
`/api/v1/meters/{id}`	GET	Unified meter details (nameplate specifications, coordinates, and healed network hierarchy).
`/api/v1/meters/{id}/energy`	GET	7-day interval energy readings (337 half-hourly points) + computed metrics: net kWh, voltage min/avg/max, power factor.
`/api/v1/hierarchy/tree`	GET	Complete electrical network topology tree: Discom → Zone → Circle → Division → Substation → Feeder → DT with meter counts.
`/api/v1/hierarchy/transformers`	GET	List of all 40 Distribution Transformers with rated kVA capacity, feeder code, and connected meters count.
`/api/v1/analytics/stats`	GET	Fleet-wide statistics: status breakdown, make distribution, phase distribution, and total transformer capacity (8,601 kVA).
`/, /dashboard`	GET	Modern web dashboard: interactive Leaflet map with color-coded pins, search table, hierarchy tree, and Chart.js energy visuals.
`/docs, /openapi.json`	GET	Interactive Swagger UI and exported OpenAPI 3.1 specification.

4. Engineering Reflection (REFLECTION.md Summary)

Q1: What assumptions did you make?

Assumed the portal is read-only and static enough that a 300s TTL cache with background pre-warming provides an optimal trade-off between upstream server load and sub-millisecond API response latency. Assumed 1-to-1 canonical mappings between electrical entity names and codes.

Q2: Which part was most difficult, and how did you get unstuck?

Reverse-engineering the HMAC-SHA256 bulk export signing mechanism and overcoming SvelteKit CSRF origin blocking. Inspected the client JS bundle (4.2Bgc2kUI.js) to discover the canonical signing string (method\npath\nquery\ntimestamp) and replicating the exact WebCrypto signing in Python.

Q3: If you had another day, what would you improve?

Add a persistent PostGIS relational database for sub-10ms spatial queries at scale, store telemetry readings in TimescaleDB for automated anomaly detection, and implement a concurrency-throttled worker queue for batch interval fetching.

Q4: What mistake did you make while solving this?

Initialized HTTPX AsyncClient inside client __init__, causing RuntimeError: Event loop is closed during synchronous Starlette TestClient pytest runs. Fixed by dynamically acquiring the running event loop and recreating the transport while preserving authentication cookies.

Q5: If you were reviewing your own submission, what would you criticise?

Relying on the internal /portal/export endpoint could be fragile if upstream changes its HMAC scheme without notice. A truly defensive service should maintain a concurrent fallback crawler against /portal/meters/search.

5. Local Execution & Automated Verification

The repository contains full automated test coverage passing 100% of tests. The codebase runs in any standard Python environment: